Computer Networks 257 (2025) 111007

Contents lists available at ScienceDirect

Computer Networks

journal homepage: www.elsevier.com/locate/comnet

3DDPS: A traffic matrix estimation method based on three-dimensional diffusion posterior sampling

Minyue Li [a], Yan Qiao [a,b,c,*], Rongyao Hu [a], Pei Zhao [a], Junjie Wang [a], Zhenchun Wei [a,b], Xuesen Ma [a], Wenjing Li [b]

[a] School of Computer Science and Information Engineering, Hefei University of Technology, Hefei, Anhui, 230601, P.R. China
[b] State key Laboratory of Networking and Switching Technology (Beijing University of Posts and Telecommunications), Beijing, 100876, P.R. China
[c] Anhui Province Key Laboratory of Industry Safety and Emergency Technology, Hefei University of Technology, Hefei, Anhui, 230601, P.R. China

ARTICLE INFO

Keywords:
Traffic matrix estimation
Network tomography
Denoising diffusion probabilistic model

ABSTRACT

Traffic matrix (TM) estimation is an essential but high-cost task for network management. A rational way is to estimate the TMs from the low-cost link load measurements by solving a group of linear equations. However, one open challenge is these linear equations are severely ill-posed in most cases. Fortunately, the emerging deep generative models offer new advanced ways to well address the ill-posed problem. In this paper, we leverage the powerful ability of diffusion models to propose a novel TM estimation framework (named 3DDPS-TME). Different from existing generative-based TM estimation methods, our new method reconstructs the raw TM data into 3D-tensor samples and modifies the typical diffusion framework to 3D-UNet to learn the spatio-temporal correlations of TMs. Furthermore, we adopt diffusion posterior sampling (DPS) for conditional sampling to produce an unbiased TM through a single sampling process. Through extensive experiments and comprehensive comparisons with four state-of-the-art baselines, the experimental results demonstrate that our method exhibits a significant superiority in both estimation accuracy and time consumption. Particularly, using only 0.03%~10.43% computational cost of the baseline methods, our method makes an improvement of 27%~68% in terms of estimation accuracy. The codes of the experiments with the proposed methods are available at https://github.com/depositoryL/3DDPS-TME.git.

1. Introduction

The traffic matrix (TM) represents the traffic demands between all possible pairs of network nodes, which are usually mentioned as origin to destination (OD) flows [1]. It can provide critical knowledge of traffic status for solving many network management problems, such as traffic engineering [2], anomaly detection [3], and capacity planning [4].

Directly measuring the traffic between all OD pairs by counting the flow packets is the most straightforward way to build a TM. Monitoring tools, such as software-defined network (SDN) architecture [5], offer an opportunity to count the volumes of particular OD flows on each network device. However, with the continuous expansion of network scale, measuring all OD flows directly by collecting each packet trace requires extremely high administrative costs as well as computational overhead [6–8].

To alleviate the expensive measurement cost, a classical way is to measure only partial OD flows, and use the partial observations to estimate the unknown flows based on the spatio-temporal structure of TMs [9,10]. However, the performance of these methods heavily relies on the number and the locations of the known OD flows. If the number of known OD flows is low or the known OD flows are not representative enough, these methods may suffer from poor accuracy. An alternative way is to estimate the flow-level traffic from the link-level statistics by means of network tomography (NT) [11]. Rather than directly measuring the OD flows (or partial), NT-based methods estimate the TMs by solving a group of linear equations that involve the link loads measured through SNMP. However, the key problem for the NT-based TM estimation method is the linear equations are usually highly rank deficient (i.e., there is no unique solution of OD flows corresponding to the measured link loads). To tackle this problem, many researchers attempt to explore the features of TMs and transform the ill-posed inverse problem into a constrained problem. For example, Vardi et al. [12] and Tebaldi et al. [7] suppose the volumes of OD flows satisfy a Poisson traffic model, while Cao et al. [13] assume a Gaussian traffic model. Zhang et al. [14] assume that the OD flows follow an

* Corresponding author at: School of Computer Science and Information Engineering, Hefei University of Technology, Hefei, Anhui, 230601, P.R. China.
E-mail address: qiaoyan@hfut.edu.cn (Y. Qiao).

https://doi.org/10.1016/j.comnet.2024.111007
Received 6 July 2024; Received in revised form 7 December 2024; Accepted 19 December 2024
Available online 26 December 2024

underlying gravity model. However, these assumptions can hardly be true for all TMs in a variety of networks.

With the rapid progress in deep learning technologies, deep generative models, such as Variational Autoencoder (VAE) [15] and Generative Adversarial Network (GAN) [16], appear as powerful feature learning tools. By training a generative model with a group of samples, the model can automatically learn the distribution of the training samples and generate synthetic samples with almost the same distribution as the training ones. Many recent works, such as [17,18], leveraged VAE and GAN to tackle the TM estimation problem, respectively. The key idea of these methods is to first train a VAE/GAN framework with a group of historical TMs, then use the trained model to generate an estimated TM that is most consistent with the tomography equations. The results in [17,18] showed a significant improvement over past assumption-based methods. However, either VAE or GAN has its inherent model defects: VAE tends to produce unrealistic, blurry samples while the training of GAN is often unstable. The denoising diffusion probabilistic model (DDPM) is a recently proposed generative model [19], which has shown its superiority over VAE and GAN in generating realistic samples without training problems. Our former work [20] made the first attempt to leverage the powerful learning ability of DDPM to produce a much more realistic TM than the former generative-based TM estimation methods. However, due to the complicated sampling process of DDPM, the computational cost of this method is extremely high. Moreover, neither the VAE/GAN-based methods nor the DDPM-based consider the obvious spatio-temporal structure of TMs, confining the generative quality of TMs.

In this paper, we proposed a novel DDPM-based framework (named 3DDPS-TME) to significantly promote the performance of existing TM estimation methods in terms of accuracy and efficiency. Firstly, to deeply exploit the spatio-temporal structure of TMs, we model the TM samples to a sequence of 3D tensors and modify the classical DDPM learning network to a 3D-UNet framework. Secondly, to guide the DDPM framework to efficiently produce an unbiased TM that conforms with the measured link loads, we develop a fast conditional sampling mechanism based on diffusion posterior sampling (DPS), which generates the target TM in a single sampling. In summary, this paper makes the following contributions as follows.

- We develop a new DDPM-based framework for accurate and efficient TM estimation. The new framework adopts the 3D-UNet learning module that can effectively learn both the local spatio-temporal structure and the long-term distribution of TMs, and thus significantly improves the synthetic quality and the estimation accuracy.
- We design a fast conditional sampling strategy to control the model to produce an optimal TM under the given link loads. By incorporating the tomography equations into each sampling step, our framework can output an optimal TM with much-reduced sampling time.
- We demonstrate the superiority of our method with comprehensive experiments on two real-world TM datasets. The experimental results show that our method can improve the estimation accuracy of the VAE-based method by 40%∼67%, the GAN-based method by 58%∼68%, the DDPM-based method by 27%∼54% on average. Notably, the estimation time of our method only takes up 0.03%∼10.43% of these methods.

2. Related work

Traffic matrix estimation (TME) is a topic of great interest due to its importance in areas such as network management. Related research has emerged in the last two decades [6], and it continues to receive considerable attention. The mainstreaming TME methods can be roughly grouped into two categories: matrix/tensor completion (MC) and network tomography (NT).

MC-based methods utilize partially collected OD flows to estimate the unknown flows by leveraging the sparsity of TMs. Zhang et al. [9] applied the compressed sensing technique to TME and proposed a method called Sparse Regularized Matrix Factorization (SRMF) to fill in the missing values in the TM. They then extended and optimized SRMF [21] so that the method can also be applied to problems such as prediction and anomaly detection. Xie et al. conducted extensive research on tensor completion-based TME methods, including Reshape-Align scheme [22], NMMF-stream [23], and Localized Tensor Completion model (LTC) [24] to recover TM data quickly and accurately. Ouyang et al. [25] combined attention-enhanced LSTM with lightweight trilinear pooling to compute the missing traffic in TMs. In addition, [10] introduces a deep generative model GAN to learn the spatio-temporal structure of TMs and generate missing traffic based on the learnt structure. However, MC-based methods basically assume a strong spatio-temporal correlation between flows, and their performance depends on the quantity and quality of the known flows. This means that there is still a considerable cost associated with this type of approach. Otherwise, when the spatio-temporal correlation between flows is weak, or when the number of known flows is small, the accuracy of these methods significantly declined.

NT-based methods estimate the complete TM from the easily collected link loads. As no traffic flow is required to be recorded, the cost of this type of method is much lower. Vardi et al. [12] assumed that the OD flows obey a Poisson distribution and used a higher-order statistical model of the OD flows to estimate the TM. However, this assumption was not verified [6]. Zhang et al. [14] improved upon [12] by using a gravity model to learn the intrinsic connection between the OD flows, resulting in an optimal solution that satisfies the link loads. The optimal solution is obtained through an Iterative Proportional Fitting Procedure (IPFP) [13] to ensure that the estimation is reasonable. The accuracy of the NT-based methods heavily relies on the validity of their assumptions. If the assumptions are not valid, the estimation accuracy will be drastically reduced.

To improve the accuracy of estimation and eliminate reliance on assumptions, researchers have introduced deep learning (DL) models to learn the features presented in the TMs. Jiang et al. [26] introduced a back-propagation neural network (BPNN) to learn the mapping relationship between link loads and TMs. They leveraged the IPFP to obtain the output that satisfies the link loads. Zhou et al. [27] proposed MNETME, which extended the input of BPNN by using the product of the Moore–Penrose inverse of the routing matrix and the link loads. The output of the model was then optimized using the Expectation Maximization (EM) algorithm [28].

With the development of deep generative models in recent years, many generative networks have been successively applied to TME methods. Kakkavas et al. [17] proposed a TME method that utilized VAE [15] to learn a latent distribution with similar characteristics to the true TM. The learnt distribution is then used to generate an estimated TM, which is then adjusted to be consistent with the link loads. Similarly, Xu et al. [18] proposed to use GAN [16] to learn the latent distribution and estimate the TM that satisfies both the link loads and the learnt distribution. Our previous work applied the DDPM to TME estimation [20]. This work first used a preprocessing module to reduce the dimensions of the TMs, and then parameterized the noise factors in DDPM and transformed the TME problem into a gradient descent optimization problem. However, due to the thousands of times of iterations in the gradient descent, it requires expensive sampling time to generate one TM, which cannot be applied to a real-time estimation task. Furthermore, all existing generative-based methods lack the considerations of spatio-temporal correlations among the flows in the TM. Hence, their learning performance still has a large room for improvement.

3. Problem formulation and background

3.1. Traffic matrix estimation

A network topology can be represented by a directed graph $G(V, E)$, where $|V|$ denotes the number of end nodes (e.g., routes) and $|E|$ denotes the number of directed links. The traffic matrix across all OD pairs in the network can be represented by $\mathbf{X} \in \mathbb{R}^{|V|\times|V|}$. For ease of computation, $\mathbf{X}$ is often reshaped into an n-dimensional vector $X = \{x_1, x_2, \dots, x_n\} \in \mathbb{R}^{n\times 1}$, where $n = |V| \times |V|$. Let $Y \in \mathbb{R}^{m\times 1}$ represent the link loads collected on each link port using SNMP, where $Y = \{y_1, y_2, \dots, y_m\}$, and $m = |E|$. The routing matrix $\mathbf{A} \in \mathbb{R}^{m\times n}$ describes the connections between links and OD flows. When the network adopts a deterministic routing policy, the elements in $\mathbf{A}$ have binary values: $a_{ij} = 1$ means the jth flow traverses the ith link; otherwise, $a_{ij} = 0$. If a probabilistic routing policy is adopted, $a_{ij} \in [0, 1]$ represents the probability of the jth flow traverses the ith link. The correlation between link loads Y and the TM X satisfies the linear equations in Eq. (1).

$$\mathbf{A}X = Y. \tag{1}$$

However, in most cases, the number of OD pairs (i.e., n) is much greater than the number of link loads (i.e., m), resulting in a highly rank-deficient equation system. Therefore, although Eq. (1) provides the mapping relationship between link loads Y and X, the TM can be hardly recovered exactly from the link loads Y without additional knowledge.

The state-of-the-art methods leverage the emerging generative model to automatically learn the distribution of TM, avoiding the unrealistic assumptions. The learnt distribution is used as the additional knowledge to obtain a unique solution in Eq. (1). However, all existing generative-based methods suffer from high computational cost in adjusting the produced TM to satisfy Eq. (1). Moreover, these methods lack the considerations of the spatio-temporal dependencies in TM, leading to a suboptimal solution.

Hence, in this paper, we aim to develop a strategy that takes the spatio-temporal dependencies of TM into account and can efficiently produce the optimal TM that satisfies both Eq. (1) and the distribution learnt by the powerful generative model (i.e., DDPM).

3.2. Denoising diffusion probabilistic models (DDPMs)

DDPMs are one class of latent variable models inspired by nonequilibrium thermodynamics [19]. DDPM applies a forward process to an original image or data x_0 by adding randomly sampled Gaussian noise ϵ. Given a variance schedule $\beta = \{\beta_1, \beta_2, \dots, \beta_T\}$, a Gaussian-noise-like data x_T is generated iteratively by adding a sequence of noise to x_0. The reverse process is executed by constantly reducing the noise of x_T until the image is completely restored. The reverse process is how DDPMs generate the samples.

The forward process of adding noise from x_{t-1} to x_t can be formulated by:

$$\begin{aligned} &x_t = \sqrt{\beta_t}\epsilon_t + \sqrt{1-\beta_t}x_{t-1}, \\ &0 < \beta_1 < \beta_2 < \cdots < \beta_t < \cdots < \beta_T < 1, \end{aligned} \tag{2}$$

where ϵ_t denotes the Gaussian noise added to x_{t-1} in the tth diffusion process, and $\beta = \{\beta_1, \beta_2, \dots, \beta_T\}$ is a variance schedule which controls the weighting of the noise. The posterior $p(x_{1:T}|x_0)$ (i.e., the forward process), is fixed to a Markov chain shown as:

$$\begin{aligned} p\left(x_{1:T} \mid x_0\right) &= \prod_{t=1}^{T} p\left(x_t \mid x_{t-1}\right), \\ p\left(x_t \mid x_{t-1}\right) &= \mathcal{N}\left(x_t; \sqrt{1-\beta_t}x_{t-1}, \beta_t\mathbf{I}\right). \end{aligned} \tag{3}$$

With Eq. (2), we can further derive the relationship between x_t and x_{t-2}, which can be formulated by:

$$\begin{aligned} x_t =& \sqrt{\beta_t}\epsilon_t + \sqrt{(1-\beta_t)\beta_{t-1}}\epsilon_{t-1} \\ &+ \sqrt{(1-\beta_t)(1-\beta_{t-1})}x_{t-2}. \end{aligned} \tag{4}$$

For the ease of notation, let $\alpha = 1 - \beta$. Since both ϵ_t and ϵ_{t-1} are Gaussian noise, Eq. (4) can be expressed using the reparameterization trick as:

$$x_t = \sqrt{1-\alpha_t\alpha_{t-1}}\epsilon + \sqrt{\alpha_t\alpha_{t-1}}x_{t-2}, \tag{5}$$

where $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ is the standard Gaussian noise.

Iteratively, after summarizing and generalizing, x_t can be calculated directly from the original data x_0 by:

$$x_t = \sqrt{1-\bar{\alpha}_t}\epsilon + \sqrt{\bar{\alpha}_t}x_0, \tag{6}$$

where $\bar{\alpha} = \prod_{t=1}^{T}\alpha_t$.

The reverse process, also known as denoising, can be formulated by a Markov chain as well:

$$p(x_{0:T}) = p(x_T)\prod_{t=1}^{T} p(x_{t-1}|x_t). \tag{7}$$

The noisy data $x_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ is usually randomly generated during the actual training and sampling process.

It is demonstrated in [19] that, given x_t and x_0, the distribution of x_{t-1}, can be formulated as follows:

$$p(x_{t-1}|x_t, x_0) = \mathcal{N}(x_{t-1}; \widetilde{\mu}_t(x_t, x_0), \widetilde{\beta}_t\mathbf{I}), \tag{8}$$

$$\begin{aligned} \widetilde{\mu}_t(x_t, x_0) &= \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\epsilon\right), \\ \widetilde{\sigma}_t &= \frac{(1-\bar{\alpha}_{t-1})(1-\alpha_t)}{1-\bar{\alpha}_t}, \end{aligned} \tag{9}$$

where $\widetilde{\mu}_t(x_t, x_0)$ and $\widetilde{\sigma}_t$ in Eq. (9) are the expectation and variance of the normal distribution, respectively.

In summary, DDPMs can learn to predict the noise ϵ from x_t to x_{t-1}, and then generate samples x_0 by iteratively sampling. The high generating quality of DDPM is widely recognized in various fields, such as images [29,30] and signals [31,32].

4. Methodology

In this section, we first provide an overview of our 3DDPS-TME framework and then describe each critical component in detail.

4.1. Overview

The framework of our 3DDPS-TME method is presented in Fig. 1. The implementation of our framework can be divided into three main phases, namely data preprocessing, 3D-UNet training, and conditional sampling. As illustrated in Fig. 1, we first use sliding window to transform the raw TM data $\mathbf{X}$ into 3D tensors $\mathcal{X}$, which are subsequently employed as the initial data $\mathcal{X}_0$ in the DDPM forward and reverse processes. After that, we train the 3D-UNet with the tensor samples to perform the reverse denoising process on the noise data $\mathcal{X}_T$. After model training, we apply a conditional posterior sampling on each denoising result $\mathcal{X}_t$ incorporating the link loads $\mathbf{Y}$ to produce a corrected result $\mathcal{X}'_t$. Finally, in the last denoising step, the output $\hat{\mathcal{X}}_0$ is the estimation of the TMs that satisfy both the link loads measurements and the distribution learnt by DDPM.

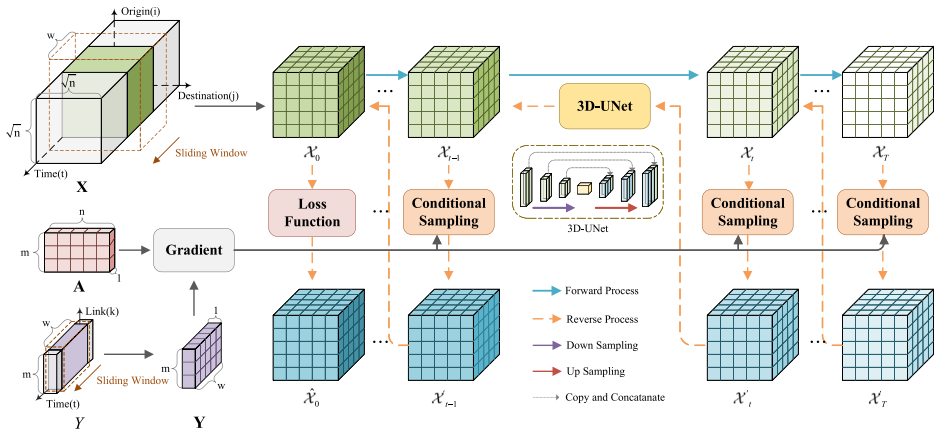


Fig. 1. Framework of proposed 3DDPS-TME. We use a sliding window to reshape the 2D TM data $\mathbf{X}$ into a 3D tensor $\mathcal{X}$, which is then used as the training data $\mathcal{X}_0$ for 3D-UNet. Then we use the link loads $\mathbf{Y}$ as the conditions to generate corresponding synthetic TMs $\hat{\mathcal{X}}_0$ from the noisy data $\mathcal{X}_T$ through conditional sampling steps performed by the trained 3D-UNet.

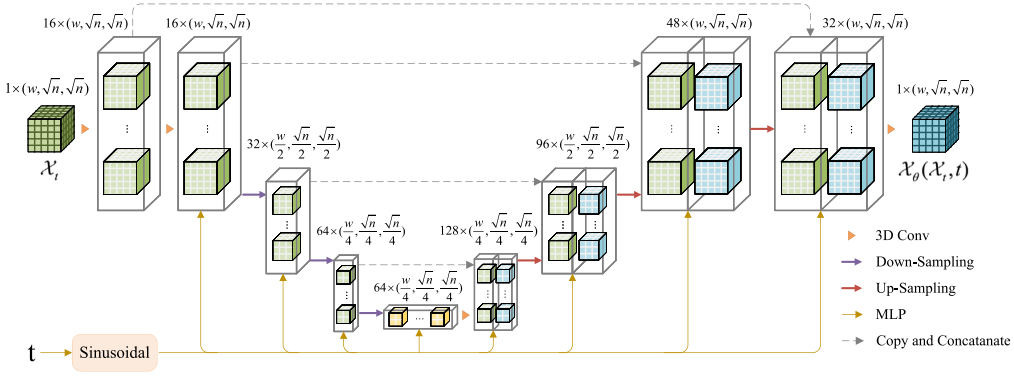


Fig. 2. Model architecture of 3D-UNet. The 3D-UNet consists of up-sampling and down-sampling. The down-sampling employs 3D-CNN-based blocks to compress the input noisy data $\mathcal{X}_t$, while the up-sampling module utilizes transposed convolution and skip connections for multi-scale feature fusion. The number and shape of the features following each processing step are shown above the blocks.

4.2. Data preprocessing

In most networks, the TMs often present obvious spatio-temporal features. In order to facilitate the learning of spatio-temporal correlations between the flows in TM, we introduce the time dimension through a sliding window, whose size w is a hyperparameter, typically taken as the number of TMs collected during a period of time. We use the sliding window to reshape the TM samples into 3D tensors, where each tensor is represented as $\mathcal{X} \in \mathbb{R}^{w\times|o|\times|d|}$, in which w is the window size, o denotes the set of original nodes, and d denotes the set of destination nodes. Generally, when there are n OD-pairs in the TM, the shape of $\mathcal{X}$ is $(w, \sqrt{n}, \sqrt{n})$. The first dimension of $\mathcal{X}$ allows the model to learn the temporal correlations in TM, while the second and third dimensions, respectively reflect the origin and destination of each flow in the network, enforcing the model to learn the spatial correlations between the flows.

After transforming the TM samples into 3D tensors, we redefine Eq. (1) to adapting the correlations between the 3D samples $\mathcal{X}$ and the link loads $\mathbf{Y}$:

$$\mathbf{A}\mathcal{X} := \mathbf{Y}, \tag{10}$$

$$\mathbf{Y}_{ki} = \sum_{q}^{\sqrt{n}} \sum_{p}^{\sqrt{n}} \mathbf{A}_{ip}\mathcal{X}_{kpq}, \tag{11}$$

where $\mathbf{Y}_{ki}$, $\mathbf{A}_{ij}$, and $\mathcal{X}_{kij}$ represent the elements in link loads $\mathbf{Y}$, routing matrix $\mathbf{A}$, and TMs $\mathcal{X}$, respectively. More specifically, the product of matrix $\mathbf{A} \in \mathbb{R}^{m\times n}$ and tensor $\mathcal{X} \in \mathbb{R}^{w\times\sqrt{n}\times\sqrt{n}}$ is calculated in accordance with the formula presented in Eq. (11), resulting in matrix $\mathbf{Y} \in \mathbb{R}^{w\times m}$.

4.3. DDPM with 3D-UNet

The UNet [33] is a classical convolutional neural network framework with a symmetrical U-shaped architecture, which was originally applied in the field of medical image segmentation. The UNet is following an encoder–decoder paradigm, except that some of the features maps output by each layer in the encoder are directly connected to the corresponding layer in the decoder through jump connections. The encoder part of the UNet, which can be considered as a contracting path, progressively compresses the input data into smaller feature maps through multilayered convolution and pooling operations while preserving high-level semantic features. The decoder part of the UNet, which can be considered as an expanding path, then combines the features of the encoder and gradually recovers the dimensions of the data. The architecture of the UNet facilitates the capture of both global and local features while processing image generation and reconstruction tasks, thereby enhancing the accuracy of the results. Its capacity to efficiently recover data corrupted by noise, coupled with its scalability, contributes to its widespread adoption in DDPMs.

The classical DDPMs adopt a 2D-UNet framework to predict the noises in the backward denoising processes. In order to facilitate the DDPM to learn the spatio-temporal feature of the tensor samples, we modify the 2D-UNet architecture in DDPM to 3D-UNet and make essential adjustments to improve the performance of learning.

The 3D-UNet initially executes the forward process of the DDPM on the preprocessed 3D tensor, thereby obtaining the corresponding corrupted data. According to Eqs. (2) and (6), the forward process of

DDPM in our study can be formulated by:

$$\begin{aligned}\mathcal{X}_t &= \sqrt{\beta_t}\epsilon_t + \sqrt{1-\beta_t}\mathcal{X}_{t-1} \\ &= \sqrt{1-\bar{\alpha}_t}\epsilon + \sqrt{\bar{\alpha}_t}\mathcal{X}_0,\end{aligned} \tag{12}$$

where $\mathcal{X}_t$ here denotes the corrupted data obtained by the addition of noise to the original TMs $\mathcal{X}_0$. Subsequently, the 3D-UNet performs noise reduction on $\mathcal{X}_t$, i.e., the reverse process of DDPM, leveraging the learned spatio-temporal correlations and distribution characteristics of the TM.

The architecture of 3D-UNet is shown in Fig. 2. Our 3D-UNet can be divided into two main modules, down-sampling and up-sampling, each of which comprises multiple 3D-CNN-based stacked ResNet blocks. The two modules are analogous to the contracting and expansive paths of the UNet [33], respectively. The transition between the two modules is also connected by two ResNet blocks. Each ResNet block contains two $1 \times 3 \times 3$ convolutional layers with padding of (0, 1, 1), using Sigmoid Gated Linear Unit (SiLU) [34] as the activation function. Considering the fact that the diffusion time step t is one of the key factors in the inverse process of the DDPM, we encode t using the sinusoidal embedding [35], and treat it as an additional input of the ResNet blocks. In the ResNet block, the embedded t is fused with the noisy sample $\mathcal{X}_t$ via a multilayer perceptron (MLP), which is then used for subsequent operations such as convolution and activation. This enables the model to achieve more precise noise reduction in accordance with the progression of the diffusion process.

The down-sampling module compresses the input information through the convolutional process in ResNet blocks and gradually extracts the high-level abstract features. In order to compensate for the degradation of the quality of the latent representation resulting from continuous down-sampling steps, we first extend the input noisy sample $\mathcal{X}_t$ to 16 channels by a $1 \times 5 \times 5$ 3D convolution with padding of (0, 2, 2), and then feed it into the down-sampling module for the corresponding processing. In the process of down-sampling, the spatio-temporal dimension of $\mathcal{X}_t$ is gradually reduced while the number of extracted features increases, as shown in details in Fig. 2.

In contrast, the up-sampling module employs multi-scale feature fusion through transposed convolution and jump connection from the down-sampling module, thereby facilitating the completion of noise reduction and restoration of the input data. After both down-sampling and up-sampling processes on $\mathcal{X}_t$ have been completed, we compress it back to a single channel using a $1 \times 1 \times 1$ convolution to obtain the final model output $\mathcal{X}_\theta(\mathcal{X}_t, t)$.

In our method, the denoising process of 3D-UNet can be summarized as the generation of denoised samples $\mathcal{X}_\theta(\mathcal{X}_t, t)$ based on $\mathcal{X}_t$ and t. In order to enable the 3D-UNet to accurately learn both the long-term distributional pattern and the short-term spatio-temporal correlations of TMs, we penalize the deviations between the denoised tensor $\mathcal{X}_\theta(\mathcal{X}_t, t)$ and the original traffic tensor $\mathcal{X}_0$. Hence, the loss function L_{diff} can be formulated by:

$$L_{diff} = \left\| \mathcal{X}_\theta(\mathcal{X}_t, t) - \mathcal{X}_0 \right\|, \tag{13}$$

where $\|\cdot\|$ can be L1 or L2 norm.[1] In our experiments, we employed the L1 norm to optimize our model.

The pseudo-code of the training algorithm for the DDPM with 3D-UNet is presented in Alg. 1. It takes as input the maximum number of training epochs I_{max}, the diffusion steps T_{diff}, the window size w, and the training data $\mathbf{X}$. It outputs the trained 3D-UNet U_θ. In Alg. 1, it first calculates the number of training data T_{train} according to the number of TMs T_{data} in the training dataset and the window size w (line 1). Subsequently, it reconstructs TMs into 3D tensors utilizing the sliding window (line 2∼4). After data preprocessing, it initializes

[1] For a 3D tensor $\mathcal{X} \in \mathbb{R}^{I \times J \times K}$, its L1 norm is $\|\mathcal{X}\|_1 = \sum_{i=1}^{I}\sum_{j=1}^{J}\sum_{k=1}^{K}\left|\mathcal{X}_{ijk}\right|$, and the L2 norm is $\|\mathcal{X}\|_2 = \sqrt{\sum_{i=1}^{I}\sum_{j=1}^{J}\sum_{k=1}^{K}\mathcal{X}_{ijk}^2}$.

Algorithm 1 Training of DDPM with 3D-UNet

Input: I_{max}, T_{diff}, w, $\mathbf{X} = \{\mathbf{X}(1), \mathbf{X}(2), \ldots, \mathbf{X}(T_{data})\}$
Output: Trained U_θ

1: $T_{train} \leftarrow T_{data} - w + 1$
2: **for** $k = 1, \ldots, T_{train}$ **do**
3: $\quad \mathcal{X}(k) \leftarrow \{\mathbf{X}(k), \mathbf{X}(k+1), \ldots, \mathbf{X}(k+w-1)\}$
4: **end for**
5: $U_\theta \leftarrow Initial()$
6: $i \leftarrow 0$
7: **while** $i < I_{max}$ **do**
8: $\quad$ **for** $j = 1, \ldots, T_{train}$ **do**
9: $\quad\quad \mathcal{X}_0 \leftarrow \mathcal{X}(j)$
10: $\quad\quad t \sim Uniform(1, \ldots, T_{diff})$
11: $\quad\quad \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
12: $\quad\quad \mathcal{X}_t \leftarrow \sqrt{1-\bar{\alpha}}\epsilon + \sqrt{\bar{\alpha}}\mathcal{X}_0$
13: $\quad\quad \mathcal{X}_\theta(\mathcal{X}_t, t) \leftarrow U_\theta(\mathcal{X}_t, t)$
14: $\quad\quad L_{diff} \leftarrow \left\|\mathcal{X}_\theta(\mathcal{X}_t, t) - \mathcal{X}_0\right\|$
15: $\quad\quad U_\theta \leftarrow GradientDescent(L_{diff}, U_\theta)$
16: $\quad\quad i \leftarrow i + 1$
17: $\quad$ **end for**
18: **return** U_θ

the parameters θ in 3D-UNet U_θ with random values (line 5). Then it updates the parameters θ in U_θ with I_{max} epochs (line 6∼17). In each epoch, Alg. 1 first treats each training data as the original data $\mathcal{X}_0$ (line 9). Then it selects a random integer from $[1, T_{diff}]$ as the current diffusion step t of $\mathcal{X}_0$ (line 10). After that, it utilizes the randomly generated Gaussian noise ϵ (line 11) to obtain the noisy sample $\mathcal{X}_t$ according to Eq. (12) (line 12). Afterwards, it treats $\mathcal{X}_t$ and t as the input of the 3D-UNet U_θ to obtain the model output $\mathcal{X}_\theta(\mathcal{X}_t, t)$, and calculates L_{diff} according to Eq. (13) (line 13∼14). Finally, Alg. 1 updates the parameters θ in U_θ through gradient descent (line 15).

After training, by inputting a Gaussian noisy $\mathcal{X}_t$ and the diffusion time step t, the 3D-UNet can utilize the learnt model parameters to perform denoising on $\mathcal{X}_t$ and generate the denoised sample $\mathcal{X}_\theta(\mathcal{X}_t, t)$ as an estimation of $\mathcal{X}_0$.

4.4. Conditional posterior sampling

After model training, the DDPM with 3D-UNet is capable of generating synthetic TMs that have the similar distribution and spatio-temporal pattern to real TMs. Our ultimate goal is to generate the optimal TM $\hat{\mathcal{X}} = \{\mathbf{X}_1, \mathbf{X}_2, \ldots, \mathbf{X}_w\}$ that can be mostly consistent with the corresponding link loads $\mathbf{Y} = \{Y_1, Y_2, \ldots, Y_w\}$, which can be formulated by:

$$\hat{\mathcal{X}} = \underset{\mathcal{X}}{argmin} \left\|\mathbf{Y} - \mathbf{A}\mathcal{X}\right\|_2^2. \tag{14}$$

All existing generative-based TME methods achieve this goal by searching for an appropriate sample from the noise space by hundreds or thousands of re-samplings, costing high computational resources. To tackle this problem, we introduce the Diffusion Posterior Sampling (DPS) [36] to our TME framework and develop a denoising method that can produce the optimal TM by a single time of sampling.

Motivated by the study in [37], the continuous forms of the DDPM's forward and backward processes in our work can be formulated by Itô stochastic differential equations, as shown in Eq. (15) and (16), respectively.

$$d\mathcal{X} = -\frac{\beta(t)}{2}\mathcal{X}dt + \sqrt{\beta(t)}d\mathcal{V}, \tag{15}$$

$$d\mathcal{X} = \left[-\frac{\beta(t)}{2}\mathcal{X} - \beta(t)\nabla_{\mathcal{X}_t}\log\ p_t(\mathcal{X}_t)\right]dt + \sqrt{\beta(t)}d\bar{\mathcal{V}}. \tag{16}$$

In Eq. (15), $\beta(t)$ is a continuous function with respect to time $t \in [0, T]$, while $\mathcal{V}$ is a standard Wiener process of the same dimension as $\mathcal{X}$. In Eq. (16), $d\bar{\mathcal{V}}$ corresponds to the standard Wiener process running backward. The time-dependent score function $\nabla_{\mathcal{X}_t} \log\ p_t(\mathcal{X}_t)$ in the reverse process can be approximated by training a score matching network s_θ.

Building on this foundation, we implement the DPS [36] via the link loads $\mathbf{Y}$ that are correlated with the TMs $\mathcal{X}$. Conditioning on $\mathbf{Y}$ as the posterior distribution of the reverse sampling process, Eq. (16) can be rewritten by Bayes' rule as:

$$d\mathcal{X} = \left[-\frac{\beta(t)}{2}\mathcal{X} - \beta(t)\nabla_{\mathcal{X}_t} \log\ p_t(\mathcal{X}_t|\mathbf{Y})\right] dt + \sqrt{\beta(t)}d\bar{\mathcal{V}}, \tag{17}$$

$$\nabla_{\mathcal{X}_t} \log\ p_t(\mathcal{X}_t|\mathbf{Y}) = \nabla_{\mathcal{X}_t} \log\ p_t(\mathcal{X}_t) + \nabla_{\mathcal{X}_t} \log\ p_t(\mathbf{Y}|\mathcal{X}_t), \tag{18}$$

where $\nabla_{\mathcal{X}_t} \log\ p_t(\mathcal{X}_t)$ in Eq. (18) is the score function, and $\nabla_{\mathcal{X}_t} \log p_t(\mathbf{Y}|\mathcal{X}_t)$ corresponds to the likelihood. The score function involving $p_t(\mathcal{X}_t)$ can be estimated by a network $s_\theta(\mathcal{X}_t, t)$, while $\nabla_{\mathcal{X}_t} \log\ p_t(\mathbf{Y}|\mathcal{X}_t)$ is difficult to compute in a closed-form since there only exists explicit dependence between $\mathbf{Y}$ and $\mathcal{X}_0$. However, we can use this dependence to approximate $\nabla_{\mathcal{X}_t} \log\ p_t(\mathbf{Y}|\mathcal{X}_t)$.

Regarding $p\left(\mathbf{Y}|\mathcal{X}_t\right)$ as the result of the marginalization of $p\left(\mathbf{Y}|\mathcal{X}_0, \mathcal{X}_t\right)$ over $\mathcal{X}_0$, and $\mathcal{X}_t$ can be expressed by $\mathcal{X}_0$ in accordance with Eq. (12), $p\left(\mathbf{Y}|\mathcal{X}_t\right)$ can be decomposed as follows:

$$\begin{aligned} p\left(\mathbf{Y}|\mathcal{X}_t\right) &= \int p\left(\mathbf{Y}|\mathcal{X}_0, \mathcal{X}_t\right) p\left(\mathcal{X}_0|\mathcal{X}_t\right) d\mathcal{X}_0 \\ &= \int p\left(\mathbf{Y}|\mathcal{X}_0\right) p\left(\mathcal{X}_0|\mathcal{X}_t\right) d\mathcal{X}_0. \end{aligned} \tag{19}$$

In (19), the posterior mean of the conditional probability distribution $p(\mathcal{X}_0|\mathcal{X}_t)$ can be computed by:

$$\begin{aligned} \widehat{\mathcal{X}}_0' &= \frac{1}{\sqrt{\bar{\alpha}(t)}}(\mathcal{X}_t + (1-\bar{\alpha}(t))\nabla_{\mathcal{X}_t} \log\ p_t(\mathcal{X}_t)) \\ &\simeq \frac{1}{\sqrt{\bar{\alpha}(t)}}(\mathcal{X}_t + (1-\bar{\alpha}(t))s_\theta(\mathcal{X}_t, t)). \end{aligned} \tag{20}$$

Utilizing $p(\mathbf{Y}|\widehat{\mathcal{X}}_0')$ to approximate $p(\mathbf{Y}|\mathcal{X}_t)$, $\nabla_{\mathcal{X}_t} \log\ p_t(\mathbf{Y}|\mathcal{X}_t)$ can be formulated as follows:

$$\nabla_{\mathcal{X}_t} \log\ p_t(\mathbf{Y}|\mathcal{X}_t) \simeq \nabla_{\mathcal{X}_t} \log\ p_t(\mathbf{Y}|\widehat{\mathcal{X}}_0'), \tag{21}$$

$$\begin{aligned} \nabla_{\mathcal{X}_t} \log\ p_t(\mathbf{Y}|\widehat{\mathcal{X}}_0') &= \nabla_{\mathcal{X}_t} \log\left[\frac{1}{\sqrt{(2\pi)^{wn}\sigma^{2wn}}} \exp\left(-\frac{\left\|\mathbf{Y} - \mathbf{A}\widehat{\mathcal{X}}_0'\right\|_2^2}{2\sigma^2}\right)\right] \\ &= -\rho\nabla_{\mathcal{X}_t}\left\|\mathbf{Y} - \mathbf{A}\widehat{\mathcal{X}}_0'\right\|_2^2, \end{aligned} \tag{22}$$

where σ is the standard variance of $p_t(\mathbf{Y}|\widehat{\mathcal{X}}_0')$, $\mathbf{A}$ is the routing matrix, and ρ is the step size.

Hence, Eq. (18) can be rewritten as the following formula:

$$\nabla_{\mathcal{X}_t} \log\ p_t(\mathcal{X}_t|\mathbf{Y}) \simeq s_\theta(\mathcal{X}_t, t) - \rho\nabla_{\mathcal{X}_t}\left\|\mathbf{Y} - \mathbf{A}\widehat{\mathcal{X}}_0\right\|_2^2. \tag{23}$$

Substituting the surrogate function in Eq. (23) back into Eq. (17), we can obtain the approximate posterior sampling of $\mathcal{X}_t$ conditioned on $\mathbf{Y}$.

By introducing DPS to our 3D-UNet framework, the sampling process can be transformed into the adjustments on the denoised tensor $\mathcal{X}_t$ given the link loads $\mathbf{Y}$ and the routing matrix $\mathbf{A}$ (as shown in Fig. 3).

Fig. 3 illustrates the adjustments with DPS in our framework. To control the denoising process to generate a qualified TM on the condition of $\mathbf{Y}$, we adjust $\mathcal{X}_t$ using the conditioned score function based on Eq. (23). For $t = \{T, T-1, \ldots, 1\}$, the denoising process from $\mathcal{X}_t'$ to $\mathcal{X}_{t-1}$ can be formulated by:

$$\mathcal{X}_{t-1} = \frac{\sqrt{\alpha_t}(1-\bar{\alpha}_t)}{1-\bar{\alpha}_t}\mathcal{X}_t' + \frac{\sqrt{\bar{\alpha}_{t-1}}(1-\alpha_t)}{1-\bar{\alpha}_t}\widehat{\mathcal{X}}_0' + \widetilde{\sigma}_t\boldsymbol{z}, \tag{24}$$

where both ϵ and $\boldsymbol{z}$ are Gaussian noise, and $\widetilde{\sigma}_t$ is the standard deviation of $p(\mathcal{X}_{t-1} \mid \mathcal{X}_t', \widehat{\mathcal{X}}_0)$.

As demonstrated by Eq. (20), $\mathcal{X}_0$, which is comprised of real TMs, can be approximately estimated directly through $\mathcal{X}_t$. According to [38], the connection between the estimation of score function $s_\theta(\mathcal{X}_t, t)$ and noise prediction ϵ_θ in DDPM can be formulated approximately as:

$$s_\theta(\mathcal{X}_t, t) \simeq -\frac{1}{\sqrt{1-\bar{\alpha}_t}}\epsilon_\theta. \tag{25}$$

Since the output of 3D-UNet $\mathcal{X}_\theta(\mathcal{X}_t, t)$ can be regarded as a prediction of $\mathcal{X}_0$, in conjunction with Eq. (6), the relationship between $\mathcal{X}_\theta(\mathcal{X}_t, t)$ and the prediction of noise ϵ_θ can be approximated as follows:

$$\epsilon_\theta \simeq \frac{1}{\sqrt{1-\bar{\alpha}_t}}(\mathcal{X}_t - \sqrt{\bar{\alpha}_t}\mathcal{X}_\theta(\mathcal{X}_t, t)). \tag{26}$$

Substituting Eqs. (25) and (26) into Eq. (20), it can be obtained that $\widehat{\mathcal{X}}_0'$ can be approximated by the output of 3D-UNet:

$$\widehat{\mathcal{X}}_0' \simeq \mathcal{X}_\theta(\mathcal{X}_t, t). \tag{27}$$

To correct the direction of the denoising in each sampling step, we utilize $\widehat{\mathcal{X}}_0'$ to minimize $\|\mathbf{Y} - \mathbf{A}\mathcal{X}\|_2^2$. Therefore, based on Eqs. (17) and (23), the adjustment of $\mathcal{X}_t$ can be formulated by:

$$\mathcal{X}_t' = \mathcal{X}_t - \eta\nabla_{\mathcal{X}_t}\left\|\mathbf{Y} - \mathbf{A}\widehat{\mathcal{X}}_0'\right\|_2^2, \tag{28}$$

where η denotes the corresponding step size. By applying the adjustments throughout all denoising steps, the final output $\widehat{\mathcal{X}}_0$ is the optimal estimation of TMs that satisfies both the learnt TM pattern and the tomography equations. The key advantage of DPS lies not only in the fast sampling speed, it was also demonstrated in [36] that DPS can produce an unbiased sampling result within the original data domain. That means the generated TM can guarantee consistency with both the learnt TM pattern and the measured link loads.

Algorithm 2 Conditional Sampling

Input: U_θ, T_{sample}, $\mathbf{Y}$, $\mathbf{A}$, η
Output: $\widehat{\mathcal{X}}_0$
1: $\mathcal{X}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
2: **for** $t = T_{sample}, \ldots, 1$ **do**
3: $\quad\mathcal{X}_\theta(\mathcal{X}_t, t) \leftarrow U_\theta(\mathcal{X}_t, t)$
4: $\quad\widehat{\mathcal{X}}_0' \simeq \mathcal{X}_\theta(\mathcal{X}_t, t)$
5: $\quad\mathcal{X}_t' \leftarrow \mathcal{X}_t - \eta\nabla_{\mathcal{X}_t}\left\|\mathbf{Y} - \mathbf{A}\widehat{\mathcal{X}}_0'\right\|_2^2$
6: $\quad\boldsymbol{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
7: $\quad\widetilde{\sigma}_t \leftarrow \frac{\sqrt{1-\bar{\alpha}_{t-1}}\sqrt{1-\alpha_t}}{\sqrt{1-\bar{\alpha}_t}}$
8: $\quad\mathcal{X}_{t-1} \leftarrow \frac{\sqrt{\alpha_t}(1-\bar{\alpha}_t)}{1-\bar{\alpha}_t}\mathcal{X}_t' + \frac{\sqrt{\bar{\alpha}_{t-1}}(1-\alpha_t)}{1-\bar{\alpha}_t}\widehat{\mathcal{X}}_0' + \widetilde{\sigma}_t\boldsymbol{z}$
9: **end for**
10: $\widehat{\mathcal{X}}_0 \leftarrow \mathcal{X}_0$
11: **return** $\widehat{\mathcal{X}}_0$

The pseudo-code of the conditional sampling in our framework is presented in Alg. 2. Alg. 2 takes as input the trained 3D-UNet U_θ, the sampling steps T_{sample}, the link loads $\mathbf{Y}$, the routing matrix $\mathbf{A}$, and the hyperparameter η. It outputs the corresponding estimated TMs $\widehat{\mathcal{X}}_0$. In Alg. 2, it first draws a noisy sample $\mathcal{X}_T$ from a Gaussian distribution (line 1). Then it performs conditional sampling steps using link loads $\mathbf{Y}$ and routing matrix $\mathbf{A}$ with T_{sample} rounds (line 2~9). In each sampling step, Alg. 2 first approximates the posterior mean $\widehat{\mathcal{X}}_0'$ using the output $\mathcal{X}_\theta(\mathcal{X}_t, t)$ of trained 3D-UNet U_θ (line 3~4). In accordance with Eq. (28), Alg. 2 obtains the adjusted $\mathcal{X}_t'$ by modifying the value of $\mathcal{X}_t$ based on $\widehat{\mathcal{X}}_0'$ and the input conditions $\mathbf{Y}$ and $\mathbf{A}$ (line 5). After that, Alg. 2 randomly generates a Gaussian noise $\boldsymbol{z}$ (line 6) and calculates the standard variance according to Eq. (9) (line 7). Subsequently, the next sample result $\mathcal{X}_{t-1}$ can be generated through Eq. (24) (line 8). Finally, Alg. 2 takes the final sampling result $\mathcal{X}_0$ as the optimal estimation of TMs $\widehat{\mathcal{X}}_0$ conditioned on the link loads $\mathbf{Y}$ (line 10).

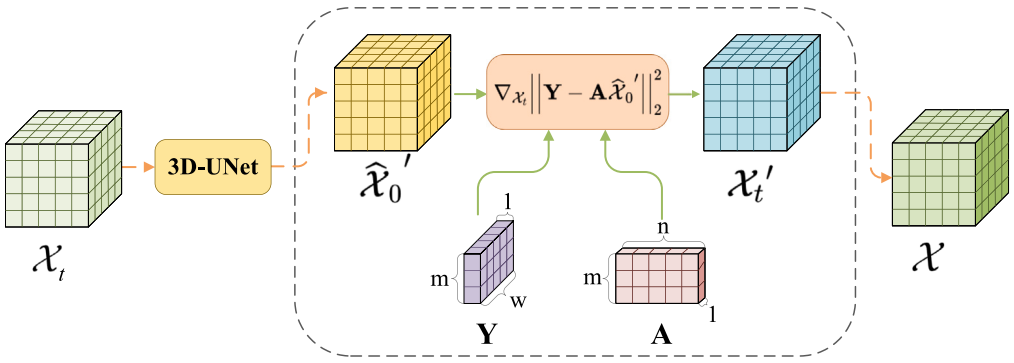


Fig. 3. The conditional sampling schematic. A posterior mean $\hat{\mathcal{X}}_0'$ is first computed from $\mathcal{X}_t$ using the trained 3D-UNet. Then the gradient of $\mathcal{X}_t$ is adjusted by the condition (i.e., link loads $\mathbf{Y}$) to a new sample $\mathcal{X}_t'$, which is more closely aligns with the condition. Finally, $\mathcal{X}_t'$ is utilized to generate $\mathcal{X}_{t-1}$ by sampling according to Eq. (24).

5. Experiments

In this section, we compare the performance of our method 3DDPS-TME with four state-of-the-art TME methods under two real-world TM datasets.

5.1. Datasets

In our experiments, two public datasets are used to verify the performance of each method: Abilene [39] and GÉANT [40]. All TM instances in Abilene were collected from the Abilene network which contains 12 routers, between March and September 2004. The training data consists of TMs from the first 15 weeks, while the testing data is from the 16th week. The GÉANT network consists of 23 routes and the TM instances in GÉANT were collected from this network from February to April 2004. The training data consists of TMs from the first 10 weeks, while the testing data is from the 11th week.

To regulate the training data onto range $[0, 1]$, all TM samples were normalized by dividing the maximum values of link loads in the datasets.

5.2. Baselines

Four state-of-the-art TME methods are employed for comparison with the 3DDPS-TME proposed in this paper. The four baselines are DDPM-TME [20], VAE-TME [17], WGAN-GP [18] and MNETME [27]. All methods are deep-learning-based TME methods that estimate the TMs from the link loads. DDPM, VAE, and GAN are the most representative deep generative models, while DDPM-TME, VAE-TME, and WGAN-GP are their respective applications, achieving state-of-the-art performance in the TME domain. MNETME is one of the most classical TME models, which was the first to introduce learning-based methods into the TME domain. The hyperparameter settings for the four baselines align with the recommended configurations originally presented. The details of these baselines are as follows.

The DDPM-TME method [20] first pretrained a preprocessing module with the objective of reducing the dimensions of TMs. Then, the DDPM was trained to learn the distribution of real TMs and generate synthetic TMs. Finally, an optimal TM was obtained by gradient-descent optimization.

The VAE-TME method [17] first trained the VAE with previously observed TMs and leveraged the trained decoder to estimate the optimal TM by solving a minimization problem in the latent space.

The WGAN-GP method [18] utilized the Generative Adversarial Network (GAN) [16] to learn the distribution of known historical TMs. Consequently, the estimation of TM was accomplished by an iterative projection-based algorithm.

The MNETME [27] employed the Moore–Penrose inverse of the routing matrix as the extended input of a Back Propagation Neural Network (BPNN). After model training, it incorporated the EM algorithm into the output of the model to obtain a qualified TM.

5.3. Metrics

We compare the performance of the TM estimation methods from three aspects: distribution similarity, estimation accuracy, and efficiency.

The distribution similarity is accessed by two visualized methods: principal component analysis (PCA) and t-distributed stochastic neighbor embedding (t-SNE). PCA reduces the dimensionality of the data by projecting it in the direction with the highest variance through a linear transformation while preserving the original structure of the data as much as possible. And t-SNE embeds the data into a low-dimensional space by preserving the relative distances between the high-dimensional data points, thereby revealing the local structure of the data.

The estimation accuracy is quantified by four different metrics: the normalized root mean absolute error (NRMSE), the normalized mean absolute error (NMAE), the spatial-related mean absolute error (SRE), and the temporal-related mean absolute error (TRE). The formulas for the four metrics are presented below respectively:

$$NRMSE = \frac{\sqrt{\frac{1}{nT_{test}}\sum_{j=1}^{T_{test}}\sum_{i=1}^{n}(x_i(j)-\hat{x}_i(j))^2}}{x_{max}}, \tag{29}$$

$$NMAE = \frac{\frac{1}{nT_{test}}\sum_{j=1}^{T_{test}}\sum_{i=1}^{n}|x_i(j)-\hat{x}_i(j)|}{\frac{1}{nT_{test}}\sum_{j=1}^{T_{test}}\sum_{i=1}^{n}x_i(j)}, \tag{30}$$

$$SRE(i) = \frac{\frac{1}{T_{test}}\sum_{j=1}^{T_{test}}|x_i(j)-\hat{x}_i(j)|}{\frac{1}{T_{test}}\sum_{j=1}^{T_{test}}x_i(j)}, \tag{31}$$

$$TRE(j) = \frac{\frac{1}{n}\sum_{i=1}^{n}|x_i(j)-\hat{x}_i(j)|}{\frac{1}{n}\sum_{i=1}^{n}x_i(j)}, \tag{32}$$

where $X(j) = \{x_1(j), x_2(j), \ldots, x_n(j)\}$, $j \in [1, T_{test}]$, represents the true TM at the jth time point, while $\hat{X}(j) = \{\hat{x}_1(j), \hat{x}_2(j), \ldots, \hat{x}_n(j)\}$ represents the corresponding predicted TM given by the model. In addition, T_{test} denotes the number of TMs used in the test.

The efficiency of the methods is measured by the training and estimating time of each method.

The main settings of our model are detailed in Table 1. The hyperparameters, including the window size w, the maximum number of training epochs I_{max}, and the loss type, are selected based on the best performance of our model. Specifically, since the down-sampling module of our 3D-UNet compresses the dimension in which w is located to $w/4$, w must be an integer multiple of 4. The mean values of the NMAE, NRMSE for our method on the two datasets when w is taken to be 4, 8, 12, and 16 are shown in Fig. 4. As illustrated in the figures, when w = 12, our method achieves the optimal performance on the majority of metrics for both datasets. All experiments were conducted on a server with 24-core CPUs at 2.4 GHz and 24 GB GPU. The results were averaged over 10 runs.

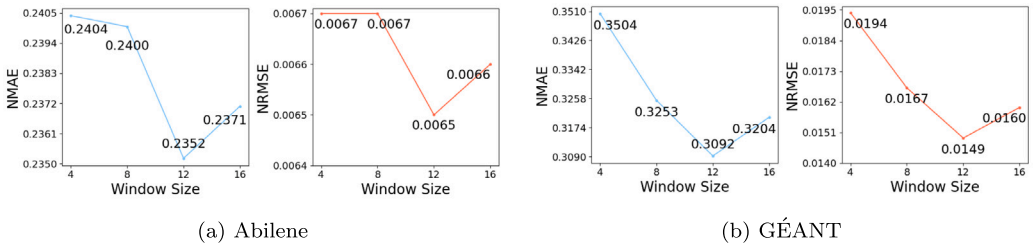


Fig. 4. Performance of 3DDPS-TME with varying window size.

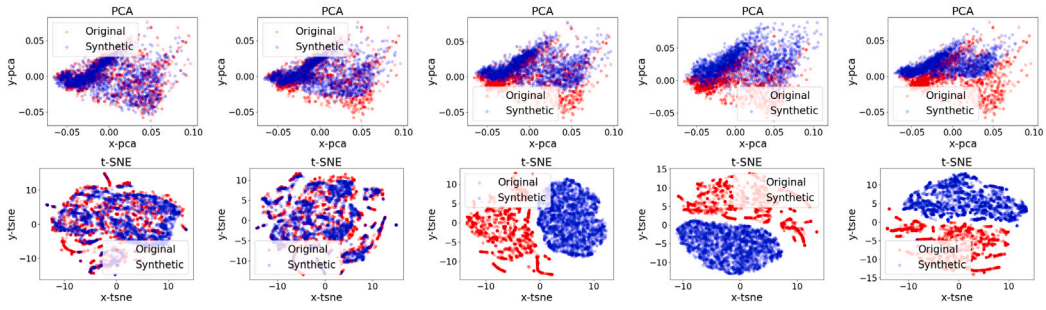


Fig. 5. PCA and t-SNE results on Abilene. The red dots denote the true TM data, and the blue dots denote the synthetic TM data of the corresponding method.

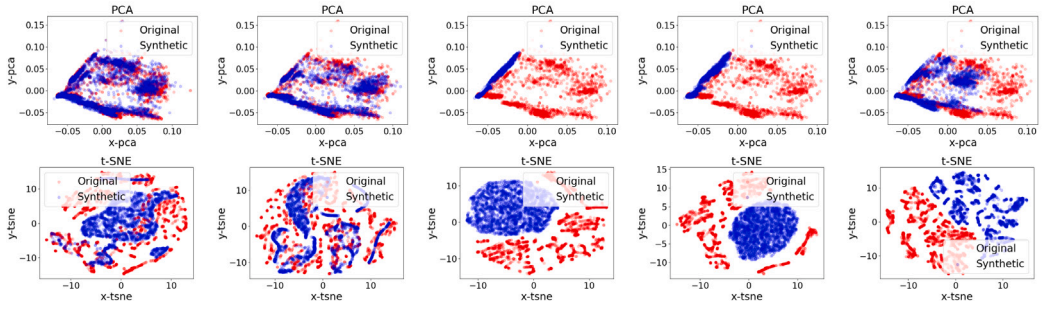


Fig. 6. PCA and t-SNE results on GÉANT. The red dots denote the true TM data, and the blue dots denote the synthetic TM data of corresponding methods.

Table 1
Hyperparameter configuration for 3DDPS-TME.

Diffusion steps	200	Loss type	L1
Sampling steps	100	Optimizer	Adam
Noise schedule	cosine	Learning rate	1.0e−5
Channels	1	Training Batch size	128
Window size	12	Estimation batch size	512
Dropout	0.0	Training epoch	15 000

5.4. Results

5.4.1. Distribution similarity

Figs. 5 and 6 visualize the similarity between the TMs produced by the five TME methods and the true TMs on the two datasets. As shown in Figs. 5 and 6, the TM data generated by our 3DDPS-TME have significantly higher overlap with the original TM data compared to other baselines. This means that our method can generate much higher quality and more realistic TM samples. We can also learn from the figures that the DDPM-based methods (3DDPS-TME and DDPM-TME) have a stronger ability to learn TM distribution than the other baselines.

5.4.2. Estimation accuracy

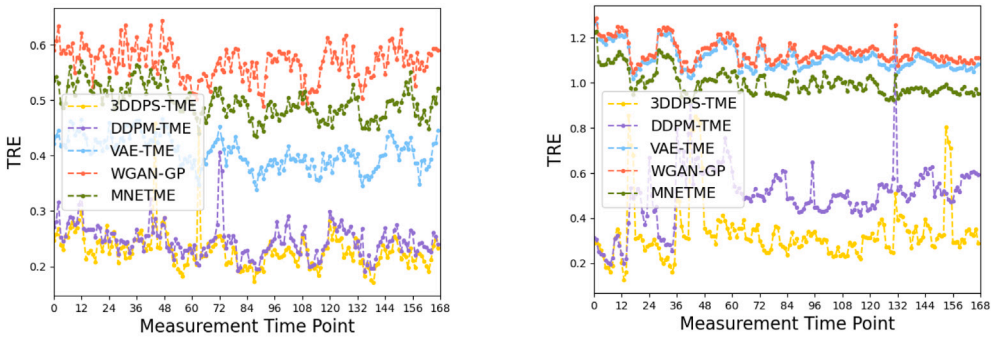
The estimation accuracy of the five methods on the two datasets is presented in Table 2. As shown in this table, our method outperforms the other baselines on the majority of metrics, which means our method has a higher accuracy and stability. In particular, 3DDSP-TME significantly outperforms the other baselines in terms of NMAE and NRMSE in most cases. It obtains the lowest Mean, Median, Std, and Max results in the metric of TRE in both two datasets. Although the SRE metric is inferior to our former work, our new method still achieves a suboptimal performance on this metric. This indicates that the incorporation of 3D-UNet can facilitate the model's ability to learn the spatio-temporal correlations of real traffic flows, especially on the temporal correlation. It is also notable that, due to the polarization of the data in the GÉANT dataset and the deficiency of the training data, all methods perform less well in this dataset. Nevertheless, our method achieves even more progress on GÉANT, indicating that our method is more resilient to skewed and deficient training data, compared to the baselines. Overall, the estimation accuracy of our method on Abilene improves the best baseline by 27% on average, while on GÉANT the accuracy can be improved by 54% on average.

5.4.3. Spatial and temporal accuracy

Fig. 7 shows the TRE trends of all methods in one week. In the figures, the testing TM data was divided into hourly units, with the 7-day

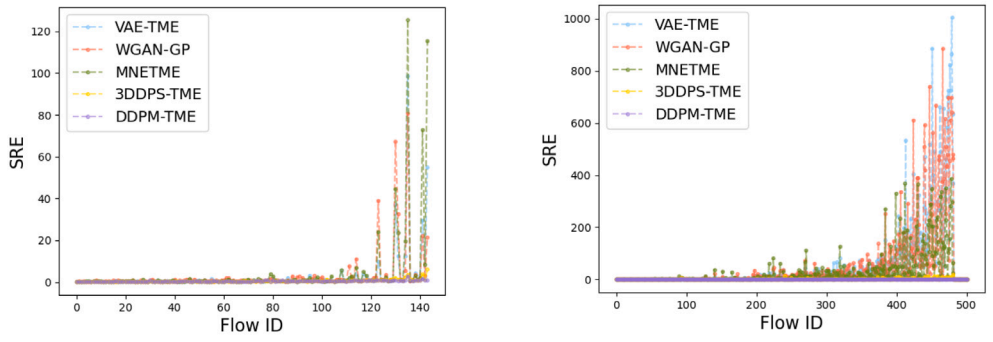
Table 2
Comparison of estimation accuracy (NMAE, NRMSE, SRE, TRE) on Abilene and GÉANT.

Dataset		Abilene				GÉANT			
Metric	Method	Mean	Median	Std	Max	Mean	Median	Std	Max
NMAE	3DDPS-TME	**0.2352**	**0.2364**	0.0042	**0.2398**	**0.3092**	**0.3117**	0.0199	**0.3392**
	DDPM-TME	0.2414	0.2411	**0.0022**	0.2448	0.4939	0.4958	0.0072	0.5010
	VAE-TME	0.4356	0.4108	0.0638	0.5416	1.1075	1.1088	**0.0048**	1.1132
	WGAN-GP	0.5875	0.5830	0.0159	0.6141	1.1446	1.1450	0.0146	1.1653
	MNETME	0.4652	0.4608	0.0240	0.4933	1.0007	0.9988	0.0063	1.0112
NRMSE	3DDPS-TME	**0.0065**	**0.0065**	**0.0001**	**0.0066**	**0.0149**	**0.0155**	0.0019	**0.0176**
	DDPM-TME	0.0089	0.0093	0.0021	0.0124	0.0322	0.0323	0.0004	0.0328
	VAE-TME	0.0109	0.0103	0.0017	0.0139	0.0456	0.0456	**0.0002**	0.0458
	WGAN-GP	0.0154	0.0153	0.0006	0.0163	0.0463	0.0464	0.0006	0.0470
	MNETME	0.0118	0.0113	0.0008	0.0130	0.0384	0.0383	0.0003	0.0389
SRE	3DDPS-TME	0.5897	0.5895	0.0213	0.6234	1.5198	1.4983	0.1217	1.7215
	DDPM-TME	**0.4903**	**0.4899**	**0.0012**	**0.4918**	**0.8300**	**0.8355**	**0.0103**	**0.8428**
	VAE-TME	2.6512	2.6459	0.0735	2.7782	51.4969	53.1058	3.4862	53.6733
	WGAN-GP	2.6172	2.7384	0.3099	2.9128	41.6893	40.4426	3.9188	48.0500
	MNETME	4.0350	3.7618	0.8252	5.6501	25.9193	25.8960	0.7126	26.9874
TRE	3DDPS-TME	**0.2370**	**0.2384**	0.0040	**0.2412**	**0.3082**	**0.3103**	0.0192	**0.3364**
	DDPM-TME	0.2439	0.2441	**0.0024**	0.2473	0.6719	0.7392	0.1002	0.7601
	VAE-TME	0.4382	0.4061	0.0640	0.5451	1.1093	1.1105	0.0048	1.1151
	WGAN-GP	0.5892	0.5855	0.0156	0.6164	1.1460	1.1464	0.0145	1.1663
	MNETME	0.4704	0.4663	0.0237	0.4984	1.0009	0.9991	**0.0063**	1.0112



(a) Abilene (b) GÉANT

Fig. 7. TREs in 168 h of five methods. The measurement results are aggregated into 168 records (i.e., once per hour).



(a) Abilene (b) GÉANT

Fig. 8. SREs of five methods. The flows on the x-axis are sorted by their averaged volumes in ascending order.

TMs divided into 168 groups. The TRE mean value was calculated for each method in each hour of the two datasets separately. Compared to other methods, our method exhibits the lowest TRE on both datasets. In particular, on the GÉANT dataset, our method demonstrates a markedly superior performance. Due to the periodicity of TM data, the results of all methods present periodicity as well. The two DDPM-based methods (3DDSP-TME and DDPM-TME) have significantly lower TRE than the other baselines. 3DDSP-TME has even lower TRE and higher stability than DDPM-TME.

Fig. 8 illustrates the SREs of the corresponding OD flows for each method on the two datasets, with the flows sorted in ascending order according to their average traffic volumes. As shown in this figure, our

Table 3
Comparison of running speed on Abilene and GÉANT.

Abilene					
Method	3DDPS-TME	DDPM-TME	VAE-TME	WGAN-GP	MNETME
Training(s)	239.58	193.08	**22.05**	1271.74	949.53
Estimation(s)	**9.64**	30 214.19	451.69	250.32	9.71
GÉANT					
Method	3DDPS-TME	DDPM-TME	VAE-TME	WGAN-GP	MNETME
Training(s)	415.34	100.24	**20.11**	873.77	668.74
Estimation(s)	10.55	15 329.05	170.30	101.11	**6.28**

method exhibits a relatively low SRE for different volumes of OD flows. It outperforms all baselines except the DDPM-TME. That indicates the introduction of the 3D-UNet structure allows the model to concentrate relatively more attention on the temporal correlation of TMs than the spatial correlation. Furthermore, all methods have a good performance on estimating small flows, which means flows with large volumes are generally difficult to estimate.

5.4.4. Efficiency

Table 3 lists the averaged training and estimating times for the five methods. The training time in this table records the average time required for each method to complete 1000 training epochs, while the estimating time records the average time required to generate 3000 TMs. As illustrated in Table 3, due to the consideration of spatio-temporal features of TMs, the training time of 3DDSP-TME is relatively longer than the original DDPM. However, the estimation time of 3DDSP-TME is dramatically reduced, taking only 0.03%/0.06% of the original form of DDPM in Abilene/GÉANT. That indicates the implementation of conditional sampling not only directs our method to generate more realistic TMs, but also significantly prompts the estimating efficiency, thereby enabling our method to real-time estimation tasks.

5.4.5. Ablation results

In order to verify the efficacy of the essential designs in our framework (i.e., 3D-UNet structure and conditional sampling) on TM estimation, we set up ablation experiments for the two modules. We use w/o 3D-UNet to denote our method applies the classical 2D-UNet structure in DDPM to learn the distribution from the regular TM samples. w/o DPS denotes our method applies the standard EM algorithm to enforce the generated TM to be consistent with the link loads, instead of the using of DPS. Table 4 lists the results of these ablation studies in the two datasets. From the tables, our method in its current form shows the best performance in almost all metrics in both datasets. That indicates the introduction of 3D-UNet and conditional sampling can greatly promote the estimation performance of our method.

In particular, compared to the 2D-UNet, the utilization of the 3D-UNet architecture that incorporates temporal and spatial dimensions can enhance the accuracy by 20.73% and 18.58% on the Abilene and GÉANT datasets, respectively. Furthermore, the integration of conditional sampling can improve the accuracy by 15.58% and 43.98% on the Abilene and GÉANT datasets, respectively. The results indicate that the incorporation of the temporal and spatial dimension into low-dimensional TM data, exemplified by the Abilene dataset, yields a more pronounced enhancement in accuracy. For high-dimensional data, such as the GÉANT dataset, the introduction of conditional sampling leads to a more substantial improvement in accuracy.

6. Discussion

Although the proposed 3DDSP-TME has shown significant efficiency and effectiveness, it is important to note that there remain certain contexts in which our method may not perform as satisfactorily.

Specifically, in 3DDSP-TME we use (28) to guide the model's sampling results, which necessitates the employment of a real routing matrix **A**. If the routing matrix is unreliable, the final sampling results will not align with the specified conditions (i.e., Eq. (14)), affecting the performance of our method. In the context of real networks, there are several factors that have the potential to render the routing matrix unreliable. For example, node failures or changes in network topology may re-schedule the routing paths of the data flows. In such a scenario, inaccuracies or errors may occur in one or multiple rows of the routing matrix **A**. Besides, for the networks that apply an adaptive routing policy to encourage load balancing, the routing matrix may change dynamically. In such networks, an accurate and stable **A** is not available.

In our future research, we will concentrate on the improvement on 3DDPS-TME for precise traffic matrix estimation in the scenarios where the routing matrix is unreliable.

7. Conclusion

In this paper, we propose a novel TM estimation framework (named 3DDPS-TME) that significantly outperforms the state-of-the-art in terms of accuracy and efficiency. On the one hand, our new method transforms the TM data into tensors and designs a 3D-UNet-based DDPM to enable the model to learn the spatio-temporal correlation of the TMs. On the other hand, we develop a conditional sampling method based on diffusion posterior sampling, which allows for fast and unbiased generation of realistic TMs that are consistent with the link loads. Through comprehensive experiments, we compared 3DDPS-TME with four state-of-the-art learning-based TM estimation methods on two real-world TM datasets. The results strongly confirm the superiority of our method. We published the source codes of our method together with the datasets in the GitHub repository.

CRediT authorship contribution statement

Minyue Li: Writing – original draft, Visualization, Supervision, Investigation. **Yan Qiao:** Writing – review & editing, Supervision, Methodology, Conceptualization. **Rongyao Hu:** Validation, Software, Resources. **Pei Zhao:** Validation, Software. **Junjie Wang:** Software, Resources. **Zhenchun Wei:** Project administration. **Xuesen Ma:** Funding acquisition. **Wenjing Li:** Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported by Open Foundation of the State Key Laboratory of Networking and Switching Technology (Beijing University of Posts and Telecommunications) (SKLNST-2024-1-02), the Fundamental Research Funds for the Central Universities of China (PA2024GDSK0095), and National Natural Science Foundation of China (NSFC) (62472140).

Data availability

The code is publicly available on GitHub, with the relevant link provided in the abstract of the paper.

Table 4
Ablation results on Abilene and GÉANT.

Dataset		Abilene				GÉANT			
Metric	Method	Mean	Median	Std	Max	Mean	Median	Std	Max
NMAE	3DDPS-TME	**0.2352**	**0.2364**	**0.0042**	**0.2398**	**0.3092**	**0.3117**	**0.0199**	**0.3392**
	w/o 3D-UNet	0.3118	0.3152	0.0102	0.3241	0.3986	0.4190	0.0381	0.4408
	w/o DPS	0.2679	0.2683	0.0071	0.2800	0.6039	0.6022	0.0206	0.6305
NRMSE	3DDPS-TME	**0.0065**	**0.0065**	**0.0001**	**0.0066**	**0.0149**	**0.0155**	0.0019	**0.0176**
	w/o 3D-UNet	0.0082	0.0082	0.0003	0.0085	0.0183	0.0202	0.0024	0.0203
	w/o DPS	0.0077	0.0078	0.0003	0.0082	0.0266	0.0270	**0.0012**	0.0282
SRE	3DDPS-TME	**0.5897**	**0.5895**	**0.0213**	**0.6234**	**1.5198**	**1.4983**	**0.1217**	**1.7215**
	w/o 3D-UNet	2.1516	2.1704	0.5367	2.8597	5.4595	4.9913	1.0077	7.4349
	w/o DPS	0.6289	0.6208	0.0236	0.6727	2.2787	2.2777	0.0921	2.4012
TRE	3DDPS-TME	**0.2370**	**0.2384**	**0.0040**	**0.2412**	**0.3082**	**0.3103**	**0.0192**	**0.3364**
	w/o 3D-UNet	0.3155	0.3209	0.0109	0.3271	0.3972	0.4181	0.0381	0.4391
	w/o DPS	0.2694	0.2687	0.0076	0.2828	0.6019	0.6004	0.0202	0.6291

References

[1] D. Jiang, Z. Xu, H. Xu, Y. Han, Z. Chen, Z. Yuan, An approximation method of origin–destination flow traffic from link load counts, Comput. Electr. Eng. 37 (6) (2011) 1106–1121.

[2] N. Wang, K.H. Ho, G. Pavlou, M. Howarth, An overview of routing optimization for internet traffic engineering, IEEE Commun. Surv. Tutor. 10 (1) (2008) 36–56.

[3] K. Xie, X. Li, X. Wang, G. Xie, J. Wen, J. Cao, D. Zhang, Fast tensor factorization for accurate internet anomaly detection, IEEE/ACM Trans. Network. 25 (6) (2017) 3794–3807.

[4] P. Tune, M. Roughan, H. Haddadi, O. Bonaventure, Internet traffic matrices: A primer, Recent Adv. Netw. 1 (2013) 1–56.

[5] D. Kreutz, F.M. Ramos, P.E. Verissimo, C.E. Rothenberg, S. Azodolmolky, S. Uhlig, Software-defined networking: A comprehensive survey, Proc. IEEE 103 (1) (2014) 14–76.

[6] A. Medina, N. Taft, K. Salamatian, S. Bhattacharyya, C. Diot, Traffic matrix estimation: Existing techniques and new directions, ACM SIGCOMM Comput. Commun. Rev. 32 (4) (2002) 161–174.

[7] C. Tebaldi, M. West, Bayesian inference on network traffic using link count data, J. Amer. Statist. Assoc. 93 (442) (1998) 557–573.

[8] R.A. Memon, S. Qazi, A.A. Farooqui, Network tomography using genetic algorithms, in: TENCON 2012 IEEE Region 10 Conference, IEEE, 2012, pp. 1–6.

[9] Y. Zhang, M. Roughan, W. Willinger, L. Qiu, Spatio-temporal compressive sensing and internet traffic matrices, in: Proceedings of the ACM SIGCOMM 2009 Conference on Data Communication, 2009, pp. 267–278.

[10] K. Xie, Y. Ouyang, X. Wang, G. Xie, K. Li, W. Liang, J. Cao, J. Wen, Deep adversarial tensor completion for accurate network traffic measurement, IEEE/ACM Trans. Netw. (2023).

[11] L. Ma, T. He, K.K. Leung, D. Towsley, A. Swami, Efficient identification of additive link metrics via network tomography, in: 2013 IEEE 33rd International Conference on Distributed Computing Systems, IEEE, 2013, pp. 581–590.

[12] Y. Vardi, Network tomography: Estimating source-destination traffic intensities from link data, J. Am. Stat. Assoc. 91 (433) (1996) 365–377.

[13] J. Cao, D. Davis, S. Vander Wiel, B. Yu, Time-varying network tomography: Router link data, J. Am. Stat. Assoc. 95 (452) (2000) 1063–1075.

[14] Y. Zhang, M. Roughan, N. Duffield, A. Greenberg, Fast accurate computation of large-scale IP traffic matrices from link loads, ACM SIGMETRICS Perform. Eval. Rev. 31 (1) (2003) 206–217.

[15] D.P. Kingma, M. Welling, Auto-encoding variational bayes, 2013, arXiv preprint arXiv:1312.6114.

[16] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, Y. Bengio, Generative adversarial networks, Commun. ACM 63 (11) (2020) 139–144.

[17] G. Kakkavas, M. Kalntis, V. Karyotis, S. Papavassiliou, Future network traffic matrix synthesis and estimation based on deep generative models, in: 2021 International Conference on Computer Communications and Networks, ICCCN, IEEE, 2021, pp. 1–8.

[18] S. Xu, M. Kodialam, T. Lakshman, S.S. Panwar, Learning based methods for traffic matrix estimation from link measurements, IEEE Open J. Commun. Soc. 2 (2021) 488–499.

[19] J. Ho, A. Jain, P. Abbeel, Denoising diffusion probabilistic models, Adv. Neural Inf. Process. Syst. 33 (2020) 6840–6851.

[20] X. Yuan, Y. Qiao, P. Zhao, R. Hu, B. Zhang, Traffic matrix estimation based on denoising diffusion probabilistic model, in: 2023 IEEE Symposium on Computers and Communications, ISCC, IEEE, 2023, pp. 316–322.

[21] M. Roughan, Y. Zhang, W. Willinger, L. Qiu, Spatio-temporal compressive sensing and internet traffic matrices (extended version), IEEE/ACM Trans. Netw. 20 (3) (2011) 662–676.

[22] K. Xie, C. Peng, X. Wang, G. Xie, J. Wen, Accurate recovery of internet traffic data under dynamic measurements, in: IEEE INFOCOM 2017-IEEE Conference on Computer Communications, IEEE, 2017, pp. 1–9.

[23] K. Xie, R. Xie, X. Wang, G. Xie, D. Zhang, J. Wen, NMMF-stream: A fast and accurate stream-processing scheme for network monitoring data recovery, in: IEEE INFOCOM 2022-IEEE Conference on Computer Communications, IEEE, 2022, pp. 2218–2227.

[24] K. Xie, X. Wang, X. Wang, Y. Chen, G. Xie, Y. Ouyang, J. Wen, J. Cao, D. Zhang, Accurate recovery of missing network measurement data with localized tensor completion, IEEE/ACM Trans. Netw. 27 (6) (2019) 2222–2235.

[25] Y. Ouyang, K. Xie, X. Wang, J. Wen, G. Zhang, Lightweight trilinear pooling based tensor completion for network traffic monitoring, in: IEEE INFOCOM 2022-IEEE Conference on Computer Communications, IEEE, 2022, pp. 2128–2137.

[26] D. Jiang, X. Wang, L. Guo, H. Ni, Z. Chen, Accurate estimation of large-scale IP traffic matrix, AEU-Int. J. Electron. Commun. 65 (1) (2011) 75–86.

[27] H. Zhou, L. Tan, Q. Zeng, C. Wu, Traffic matrix estimation: A neural network approach with extended input and expectation maximization iteration, J. Netw. Comput. Appl. 60 (2016) 220–232.

[28] G.J. McLachlan, T. Krishnan, The EM Algorithm and Extensions, John Wiley & Sons, 2007.

[29] C. Saharia, J. Ho, W. Chan, T. Salimans, D.J. Fleet, M. Norouzi, Image super-resolution via iterative refinement, IEEE Trans. Pattern Anal. Mach. Intell. 45 (4) (2022) 4713–4726.

[30] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, B. Ommer, High-resolution image synthesis with latent diffusion models, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2022, pp. 10684–10695.

[31] N. Chen, Y. Zhang, H. Zen, R.J. Weiss, M. Norouzi, W. Chan, Wavegrad: Estimating gradients for waveform generation, 2020, arXiv preprint arXiv:2009.00713.

[32] Z. Kong, W. Ping, J. Huang, K. Zhao, B. Catanzaro, Diffwave: A versatile diffusion model for audio synthesis, 2020, arXiv preprint arXiv:2009.09761.

[33] O. Ronneberger, P. Fischer, T. Brox, U-net: Convolutional networks for biomedical image segmentation, in: Medical Image Computing and Computer-Assisted Intervention–MICCAI 2015: 18th International Conference, Munich, Germany, October 5-9, 2015, Proceedings, Part III 18, Springer, 2015, pp. 234–241.

[34] P. Ramachandran, B. Zoph, Q.V. Le, Searching for activation functions, 2017, arXiv preprint arXiv:1710.05941.

[35] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A.N. Gomez, Ł. Kaiser, I. Polosukhin, Attention is all you need, Adv. Neural Inf. Process. Syst. 30 (2017).

[36] H. Chung, J. Kim, M.T. Mccann, M.L. Klasky, J.C. Ye, Diffusion posterior sampling for general noisy inverse problems, 2022, arXiv preprint arXiv:2209.14687.

[37] Y. Song, J. Sohl-Dickstein, D.P. Kingma, A. Kumar, S. Ermon, B. Poole, Score-based generative modeling through stochastic differential equations, 2020, arXiv preprint arXiv:2011.13456.

[38] Y. Zhu, K. Zhang, J. Liang, J. Cao, B. Wen, R. Timofte, L. Van Gool, Denoising diffusion models for plug-and-play image restoration, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2023, pp. 1219–1229.

[39] Y. Zhang, Abilene network topology data and traffic traces, 2004.

[40] S. Uhlig, B. Quoitin, J. Lepropre, S. Balon, Providing public intradomain traffic matrices to the research community, ACM SIGCOMM Comput. Commun. Rev. 36 (1) (2006) 83–86.

Minyue Li obtained her Bachelor's degree from Zhejiang Normal University. She is currently pursuing a Master's degree with the School of Computer Science and Information Engineering, Hefei University of Technology. Her research interests include network management and deep learning.

Yan Qiao received the Ph.D. degree in computer science from Beijing University of Posts and Telecommunications in 2012. She was a Post Doctoral Fellow with the School of Computer Engineering, Nanyang Technological University, Singapore. She is currently the associate professor with the School of Computer Science and Information Engineering, Hefei University of Technology, China. She now focuses on network monitoring, anomaly detection for time series, and artificial intelligence.

Rongyao Hu is a postgraduate student of Artificial Intelligence with the School of Computer Science and Information Engineering, Hefei University of Technology. His research interests include the detection of anomalies in multidimensional time series and the application of deep generative modeling.

Pei Zhao received a Bachelor's degree from Anhui University of Finance and Economics. Currently, he is a postgraduate student with the School of Computer Science and Information Engineering, Hefei University of Technology. His research interests focus on next-generation Internet technologies and anomaly detection.

Junjie Wang graduated from Anhui University of Science and Technology. He is currently enrolled in the Artificial Intelligence program with Hefei University of Technology, pursuing a Master's degree. His research interests include the field of Internet of Things (IoT) network technology and the application of deep learning techniques.

Zhenchun Wei received the B.S. and Ph.D. degrees from Hefei University of Technol ogy, in 2000 and 2007, respectively. He is currently an Associate Professor with the S chool of Computer Science and Information Engineering, Hefei University of Technol ogy. His research interests include Internet of Things, distributed intelligence, deep le arning and edge computing.

Xuesen Ma was born in 1976. He received the B.S. in electric engineering and Ph.D. degrees in computer science from the Hefei University of Technology, Hefei, China, in 2000 and 2016, respectively. He was a Visiting Scholar with the Research School of Computer Science, Australian National University, from 2018 to 2019. He is currently an associate Professor with Hefei University of Technology. His current research interests include computer vision and unmanned aircraft system.

Wenjing Li received the B.S. degree in computer science and technology from the Beijing Institute of Technology, Beijing, China, in 1995, and the Ph.D. degree in communication and information system from the Beijing University of Posts and Telecommunications (BUPT), Beijing, China in 2016. She is currently a Professor with the Beijing University of Posts and Telecommunication, and serves as the Director with the Key Laboratory of Network Management Research Center. Meanwhile, she is the Leader of TC7/WG1 with the China Communications Standards Association. Her research interests include wireless network management and automatic healing in SONs.